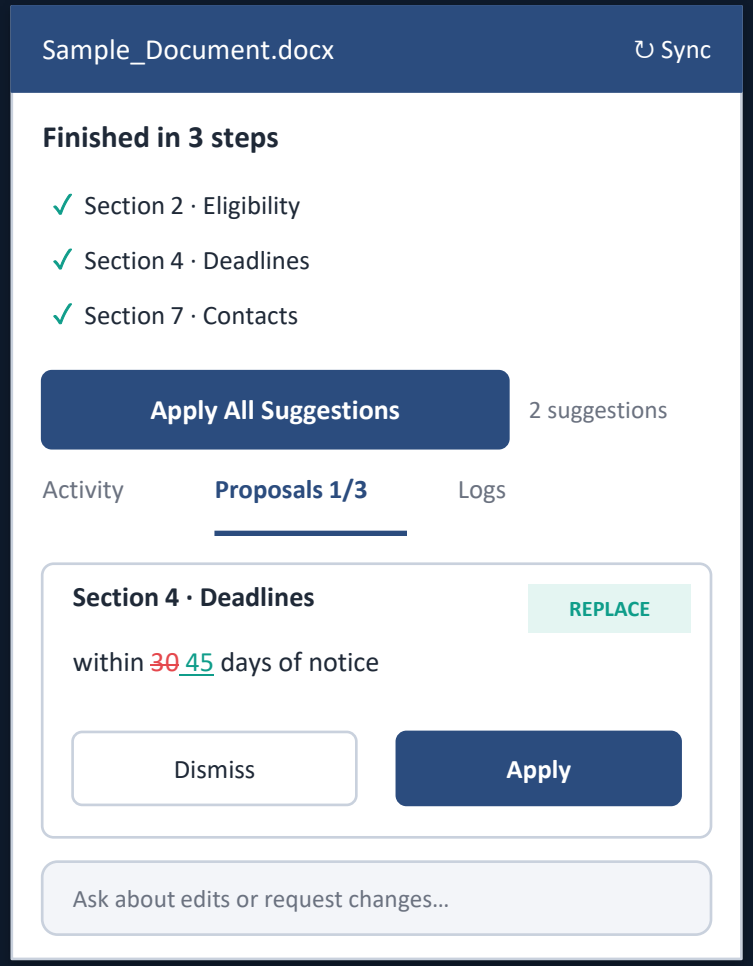
What Breaks When an AI Agent Meets Production

Lessons from running a document-editing agent on Amazon Bedrock AgentCore

Shreya R
Research Engineer 2 - AI, Thomson Reuters

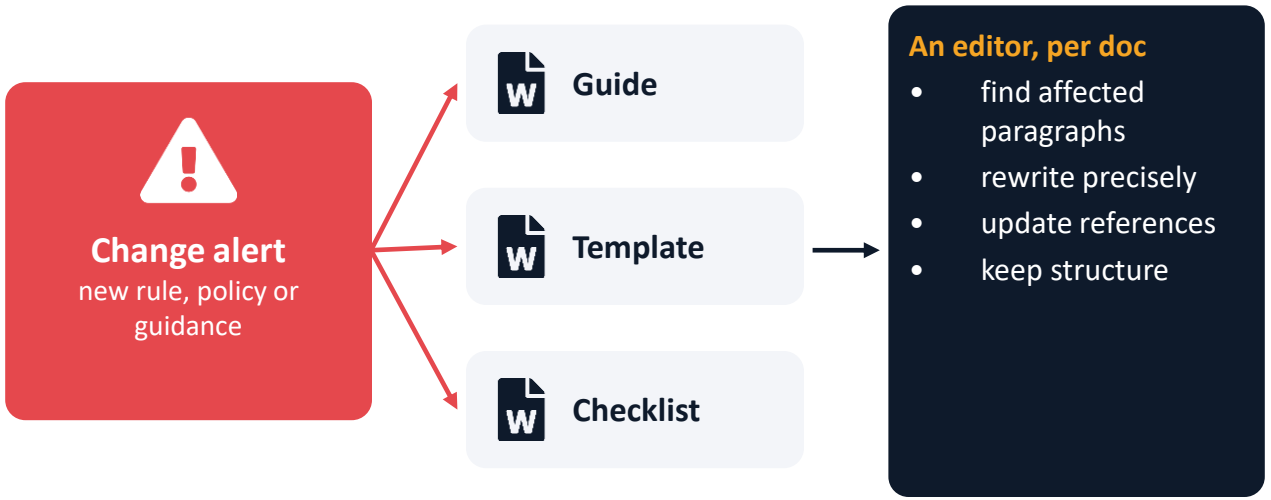


Illustrative taskpane (sample content)

THE PROBLEM

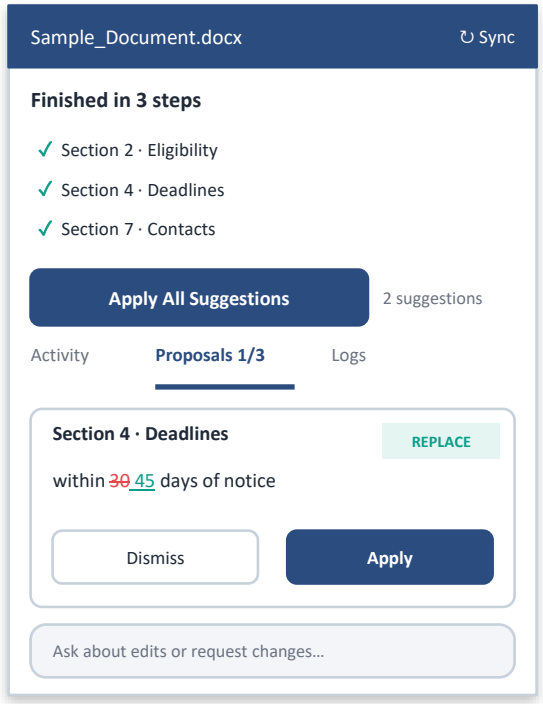
Every source change means careful, manual edits

THE PROBLEM



every week, across many documents

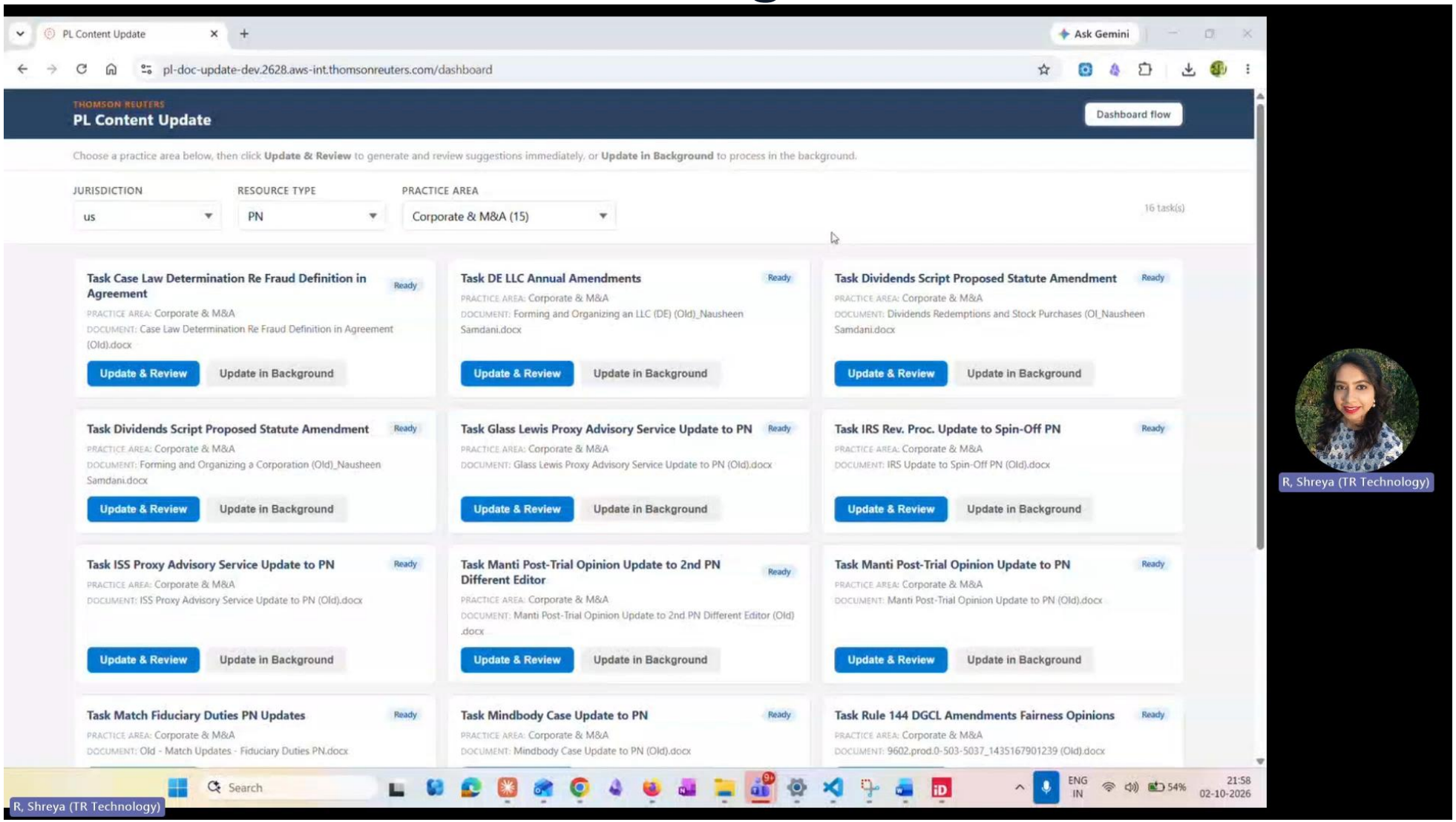
WHAT WE BUILT



1. **Word add-in**
   no new tool for editors
2. **Agent proposes**
   edits stream in live
3. **Editor decides**
   Apply → word-level Track Changes

The constraint that shapes everything: a wrong edit is worse than no edit. The agent proposes; a human decides.

Demo: from alert to Track Changes



Lesson 1: better agent design beat a bigger model

BEFORE · FACT-FIRST

Read the alert
no document context

→

Extract every fact
many per alert

→

Hunt for a home
for each fact in the doc

→

Most went nowhere
each one a risk of a wrong edit

NOW · DOC-FIRST

Read the document
no alert in sight

→

Map each section
purpose + what could belong

→

Read the alert through the map
filter while reading

→

A plan of 1–5 moves
close to what experts do



Today: a fixed 8-step pipeline with one tool-calling loop. Everything before and after that loop is plain, testable code.

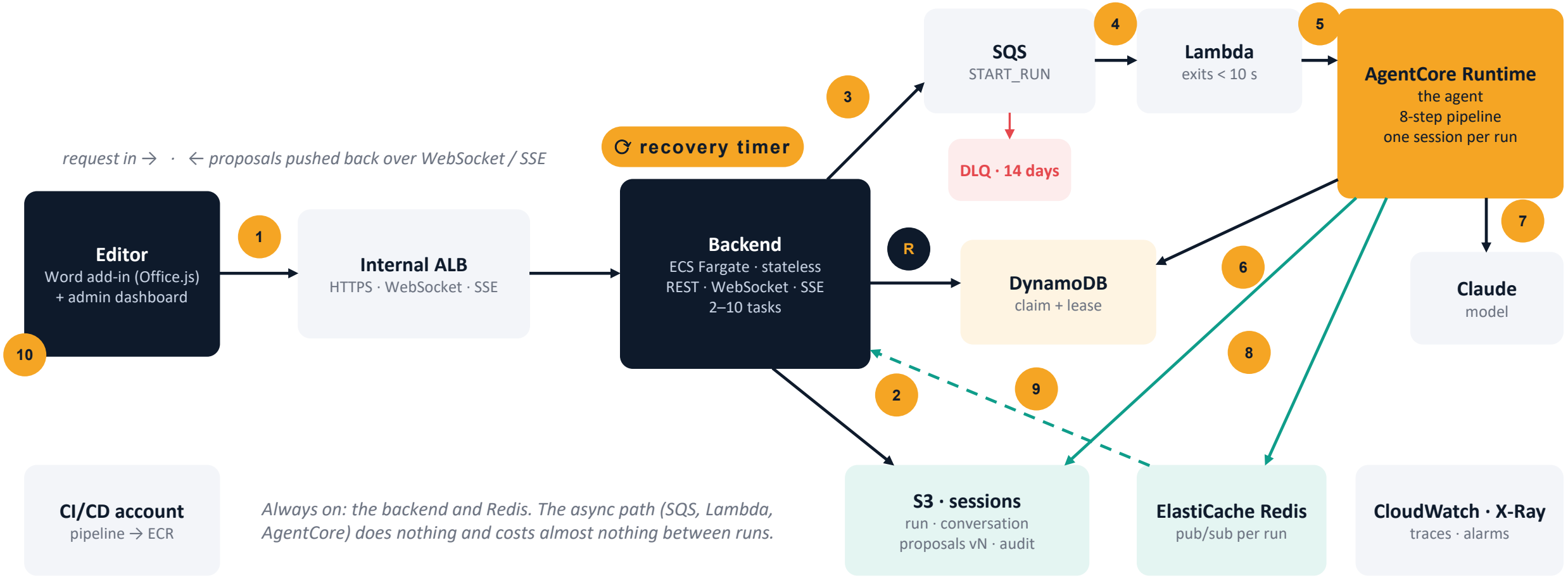
Seven questions the prototype never had to answer

	Prototype		Production asks…
1	In-process asyncio task	→	**A run is one 10–15 min conversation. Where does it run?**
2	One server, always running	→	**It runs a few times a day. Why pay all day?**
3	One process, one run	→	**SQS delivers at least once. Processing must be safe to repeat.**
4	Crash = start over by hand	→	**If the agent dies mid-run, the run recovers by itself.**
5	In-process WebSocket	→	**Live progress, with the backend on many tasks?**
6	No audit trail	→	**Every model decision traceable after the fact.**
7	Local npm start	→	**Production gets exactly what we tested, in 3 envs.**

Once the prototype works, the hard problems move beyond the model.

THE ARCHITECTURE

The complete architecture, and one run through it



1 start run 2 save state 3 queue START_RUN 4 trigger 5 invoke async 6 claim + lease 7 pipeline + Claude 8 save + publish proposal 9 push to Word 10 apply **R** recovery sweep

Lesson 2: never make a request wait for an agent

The problem

A run takes 10–15 min. Between the editor and the agent:

- ALB: 60 s default idle timeout
- proxies and firewalls add their own limits
- laptops sleep, networks drop



accepting the work is not doing the work

Why a queue, not a direct call

failed hand-offs retry automatically · failed runs land in the DLQ, never lost · the editor can disconnect and restore from S3 · backend deploys don't kill runs in flight

Settings that matter

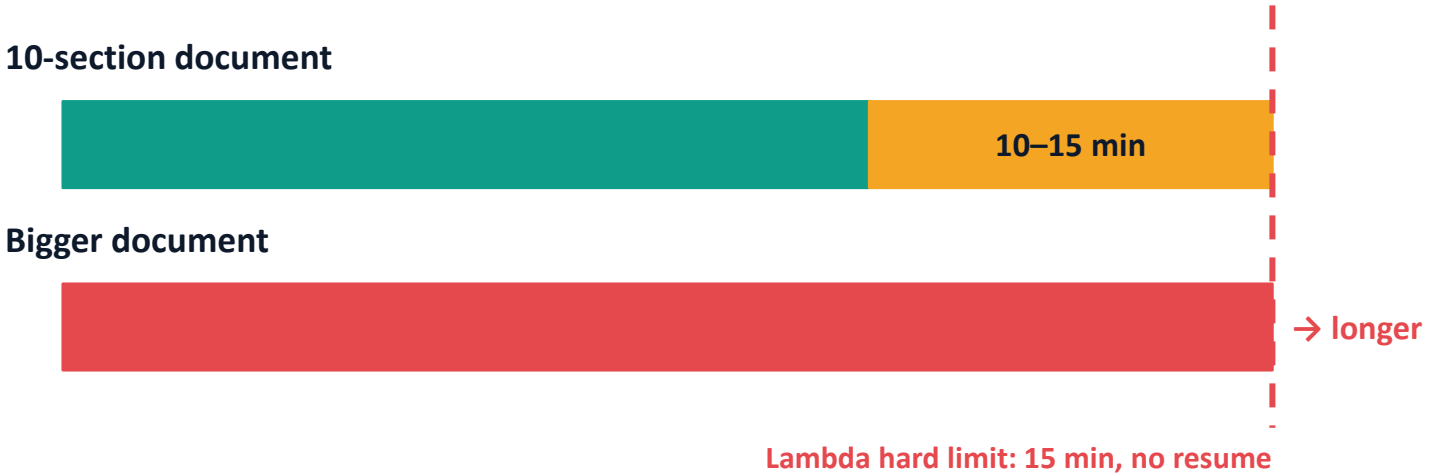
batch size 1 · visibility timeout 240 s (far above Lambda's < 10 s) · max receive count 3 → DLQ

Lesson: accept the work fast, do it somewhere else, and let the queue own retries. (Catch: it delivers at least once. See lesson 4.)

ANSWERS #1 #2

Lesson 3: pick the runtime by workload shape

One continuous model conversation per document



✗ **AWS Lambda**
15-min hard limit; a timeout loses the agent's context

✗ **Always-on ECS for the agent**
a container running 24/7 for a few runs a day

✓ **AgentCore Runtime**
async work up to 8 h per session; per-second billing, no CPU charge while waiting on the model

Lambda didn't leave; it moved. It consumes the SQS message, calls invoke_agent_runtime() asynchronously, and exits in under 10 s. The system still has always-on parts (the backend). What we avoid is an always-on agent fleet.

ANSWERS #3 #4

Lesson 4: every lock needs an expiry

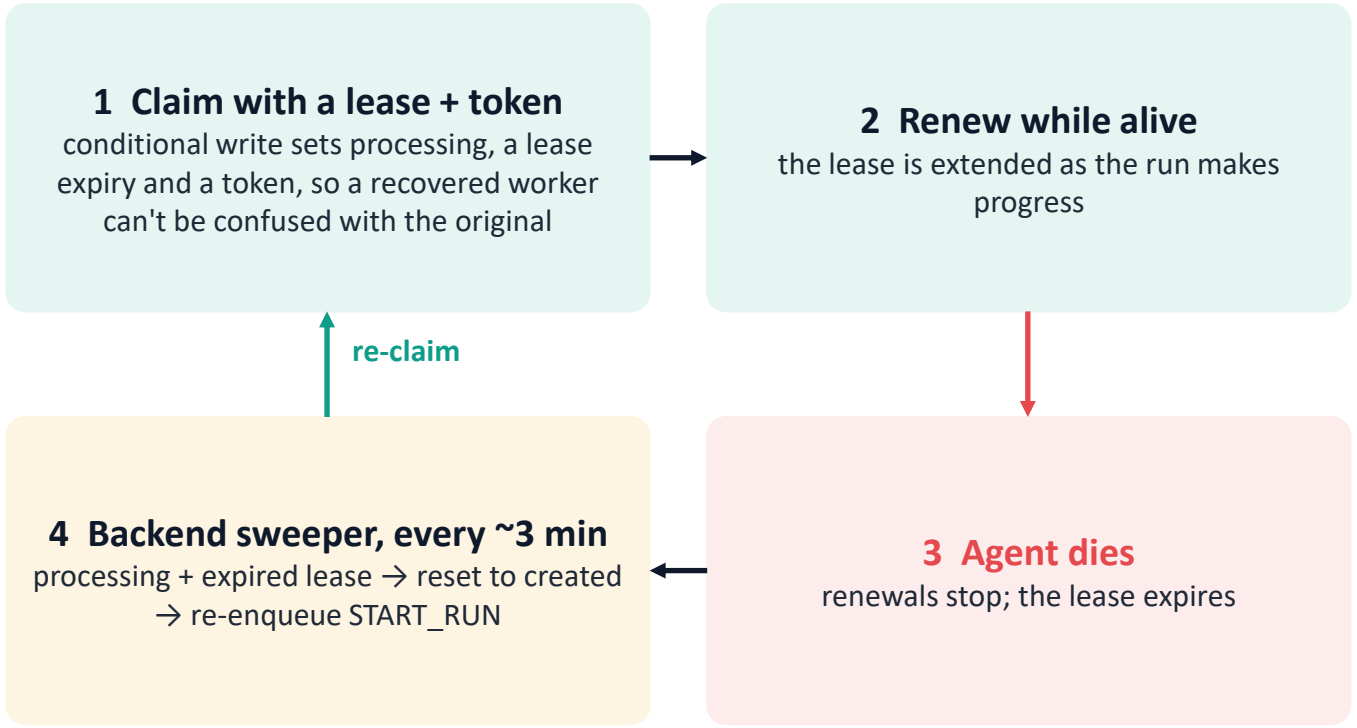
HALF 1 · SAFE TO REPEAT

SQS is at-least-once. A DynamoDB conditional write lets exactly one agent move a run `created → processing`; the other gets ConditionalCheckFailed and exits.

HALF 2 · WHAT WENT WRONG

A batch of runs sat at 'processing' for days. The first claim had no way to release a run if its worker died or got stuck. The SQS message was consumed, the lock never released, and no path was allowed to touch a locked run.

Why a timer in the backend? It's already always on (2 tasks, 24/7). No new service to operate.



Cheap recovery: finished sections already exist in S3 and are re-emitted, not regenerated. **Too many retries → error → alarm.**

Lesson 5: let the judge advise, never delete



Correct edits, deleted

by a verifier that was allowed to reject, as later confirmed by expert review

Deterministic checks can block. The model judge can only advise.
Originally a rejection removed the edit. Now the verdict and reason travel with the edit, and the editor sees both. **The human decides.**

Lesson 6: test on the real client and the real network

STORY 1 · THE CLIENT

Symptom	Mac users couldn't open documents. Word offered to 'create a local copy'. Nothing was logged.
Why it passed	Every automated test passed. It only failed for real users on one operating system.
Cause	Links were ~2,500 characters (the signing credentials alone add 900+). Word for Mac silently cuts links near 2,000.
Fix	Sign the links with a separate identity that produces a much shorter signature.

STORY 2 · THE NETWORK

Symptom	Connections failed on and off. It looked like flaky infrastructure.
Why it passed	It worked from developer machines, which take a different network path than a work laptop.
Cause	The domain sounded internal but pointed at a public CDN. Some zero-trust connectors could reach it; others only reached private addresses.
Fix	Move the front door to an internal load balancer, reachable start to finish over the path users are on.

The common thread: both passed every test we ran ourselves. Length limits, silent cutoffs and network paths only show up on the real client and the real network.

Lesson 7: the request succeeded. The agent still failed.

t = 0
SQS triggers Lambda

→

t ≈ 1 s
Lambda hands off and returns ✓
error rate 0 · latency fast

→

minutes later
agent fails inside the runtime ✗
no failed request anywhere

→

The rule
watch whether the work finished, not whether the request returned

HOW WE TRACE A FAILED RUN ON BEDROCK · everything lands in CloudWatch

1 Metrics
Is something failing?

AgentCore runtime metrics: sessions, latency, tokens, errors, throttles. Always on, and what alarms are built on. But no run-level detail.

2 Traces
Where did it fail?

OpenTelemetry spans for every model call and tool call. The failed step carries the error. Braintrust adds prompts, outputs and editor decisions.

3 Logs
Why did it fail?

Exceptions and throttling lines, queried with Logs Insights. Metric filters turn them into alarms: 'run gave up', 'agent errors'.

If the agent dies, it can't report it. Watch what goes missing:

1. **Lease stops renewing**
the run sits in 'processing'; the backend sweeper finds it

2. **Audit trail in S3**
last event: an LLM call that started and never finished

3. **Queue stops draining**
DLQ fills · 'run gave up' fires

To investigate
one run ID in every service's logs → one search shows the whole path

Lesson 7 (cont.): set up alarms and one dashboard

One CloudWatch dashboard: every signal in one place *illustrative*

System critical · OK	Critical · OK	Warning · OK
Queue: oldest message age 	**Dead-letter queue depth** 0 *healthy = empty*	**Dispatcher errors · throttles**
Agent errors CUSTOM metric filter over the runtime logs	**Runs that gave up** CUSTOM metric filter: recovery used up its retries	**Load balancer · DB · containers · cache** ALB 5xx · Database throttles · CPU / memory · Cache pressure

Critical · page now
DLQ not empty · oldest message age · dispatcher errors · ALB 5xx · DB throttles · run gave up

Warning · look today
queue depth · duration · CPU · memory · cache

Composite: system critical
red if any critical alarm is red: one answer to 'is it on fire?'

One notification topic
email / on-call

Five jobs: metrics detect, traces locate, logs explain, the dashboard diagnoses, alarms make sure somebody knows. Built as a read-only IaC stack.

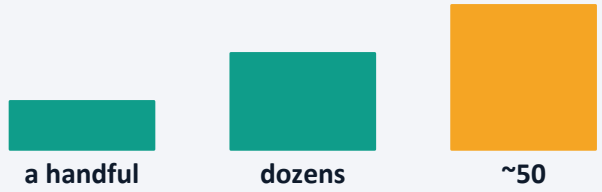
Lesson 8: break it on purpose, before users do

1 A fake client

Makes the same API calls the dashboard makes to start a run.

Then polls the status table, because the first response says nothing about the outcome: the run finishes later.

2 Step up the load



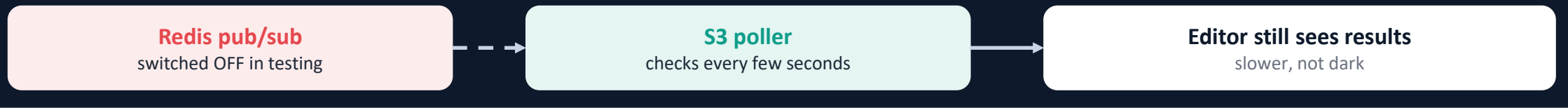
Hard cap on total runs and total spend: every fake run is a real, billed model call.

3 Read the result

- Backlog drains after a burst
- Dead-letter queue stays empty
- Runs finish, not error

The bottleneck was the model provider, not our infrastructure.

And break the fast path on purpose



Lesson: find the limits and the failure modes yourself, before users do. Plan capacity from the model's tokens per minute.

Lesson 9: build once, promote the same artifact



The bytes you tested are the bytes you ship.

AgentCore: same-account images

The pipeline copies the agent image from the CI/CD account's ECR into the app account's ECR before deploying.

Stack order via Parameter Store

The agent stack writes the runtime ARN; the orchestrator stack reads it, so the runtime exists first with no circular dependency.

Check behavior, not just code

A fixed set of known-good documents, three measures: right sections, correct edits, run-to-run consistency.

NINE LESSONS

Nine lessons, one slide

How it runs

1. Fix agent design before a bigger model
2. Never make a request wait for an agent
3. Pick the runtime by workload shape
4. Every lock needs an expiry
5. Let the judge advise, never delete

How we know it works

6. Test on the real client and the real network
7. A successful request isn't a successful agent: see it, and get told
8. Break it on purpose: load test before users do
9. Build once, promote the same artifact

For an AI agent, “is it up?” is the wrong question.

Ask: did the work finish, and would we know?

What stuck runs, a link that silently broke, and green dashboards over failing runs taught us.

Thank you

Shreya R · Research Engineer 2 - AI, Thomson Reuters

FIND ME

LinkedIn
linkedin.com/in/shreya-r-51166a227

Medium
medium.com/@shreyaaradhya06

THE THREE BLOGS BEHIND THIS TALK

1. **From POC to Production: Building an AI Document Updater with AWS Bedrock AgentCore**
   medium.com/@shreyaaradhya06/from-poc-to-production-…
2. **Production-Hardening an AI Agent: Real-Time Delivery, Observability, and CI/CD on AWS**
   medium.com/@shreyaaradhya06/production-hardening-an-ai-agent-…
3. **Shipping AI Agents: A Production Readiness Checklist**
   medium.com/@shreyaaradhya06/shipping-ai-agents-…